

Safe<T>: Automaton-Theoretic Runtime Monitoring of Blackbox Distributed Systems

Karuna Grewal
Cornell University
Ithaca, NY, USA
kgrewal@cs.cornell.edu

Abstract

Modern safety-critical systems are increasingly built from blackbox components: cloud services, third-party APIs, autonomous systems, and AI agents whose source code or internal state may be unavailable for inspection. To certify safe and secure inter-component interactions in such systems, security and compliance teams must enforce policies over sequential and nested call/return patterns in the execution, along with the data values exchanged between the components. Furthermore, the blackbox setting necessitates decoupling the policy enforcement mechanism from the system implementation. To this end, I design Safe<T>, an automaton-theoretic runtime monitoring framework for specifying and enforcing control-flow and data-aware policies over blackbox distributed systems. My technique is blackbox and non-invasive, *i.e.*, it requires no code access or implementation changes. To realize this framework, I have built a distributed runtime monitor on top of an emerging network infrastructure layer that can control inter-component communication. I have demonstrated Safe<T> on cloud microservices and am now extending it to agentic AI systems.

CCS Concepts: • Networks → Cloud computing; • Security and privacy → Logic and verification.

Keywords: Runtime Monitoring; Cloud; Security.

ACM Reference Format:

Karuna Grewal. 2026. Safe<T>: Automaton-Theoretic Runtime Monitoring of Blackbox Distributed Systems. In *Companion Proceedings of the 2026 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity (SPLASH Companion '26)*, October 4–9, 2026, Oakland, CA, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3837729.3842148>

1 Motivation

Software has become deeply embedded in high-stakes decisions: it steers autonomous systems, administers medical

care, processes financial transactions, and is increasingly acting autonomously through AI agents on behalf of users and organizations. This raises the stakes for certifying these systems as safe and compliant: they must not mishandle or leak confidential data or take risky, irreversible actions. For instance, a cloud-based healthcare application must comply with HIPAA regulations and not share personal health information without privacy protections; a personal AI assistant must not forward confidential emails to an adversary.

However, the systems that most need assurance are often the hardest to inspect. Proprietary services, third-party APIs, and agentic systems may not expose their source code or internal state. Static techniques like type system-based information flow control systems [2] are unsuitable because: (a) they require access to or modification of the system, (b) they cannot adapt to dynamic systems, like a cloud application invoking a new version of a third-party API that was rolled out while the application was running.

Runtime monitoring offers a promising alternative for enforcing safety properties from the outside by observing the system's behavior. The key opportunity is that even blackbox systems with distributed components reveal their behavior through inter-component interactions. For example, cloud applications call APIs, agentic AI systems invoke tools, and vehicular control systems issue commands over buses.

2 Problem

Existing runtime monitoring techniques are not fully suited for distributed inline monitoring of modern networked systems, where the relevant behavior is observed across component boundaries through distributed events such as agent tool calls, controller actions, and software API calls. Enforcing rich safety properties for such systems requires reconstructing the execution structure, such as the history leading to an event, the hierarchical structure of the surrounding sequence of events, causality, and the flow of sensitive data. Existing approaches rarely support this execution structure and are often difficult to deploy in practice because they often require access to code for instrumentation or to internal state for analysis, which are unavailable in many systems. These gaps motivate my central research question: **How can we specify and enforce structural and data-aware safety properties over the execution behavior of a blackbox distributed system, without accessing or modifying its source code?**



This work is licensed under a Creative Commons Attribution 4.0 International License.

SPLASH Companion '26, Oakland, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2910-2/2026/10

<https://doi.org/10.1145/3837729.3842148>

To this end, I am developing the Safe(T) framework comprising: (a) an expressive specification language for policies, (b) an automaton-theoretic enforcement mechanism, and (c) a distributed runtime monitor that is non-invasive, i.e., does not update or need access to the software code.

3 Approach

The central observation motivating my dissertation is that many modern software systems share a common design: they are composed of loosely coupled components that communicate through well-defined interfaces over network protocols like HTTP, gRPC, etc. This common structure lets me study the broader problem of enforcing inter-component communication safety properties through the concrete and widely deployed setting of microservice-based cloud applications.

I first develop Safe(T) for the microservice setting as it exposes the core technical challenges underlying my research question. Then, to highlight the broad applicability of my technique, I extend Safe(T) to agentic AI systems.

Domain 1: Cloud Microservices

Consider a cloud-native hospital application whose lab-testing workflow is decomposed across multiple components or microservices, each exposing an API: Front receives raw medical details, Enc encrypts data, Lab sends data to a third-party lab, and Log records audit information. A single request produces a *service tree*: services call other services, which may recursively call others. Consider a HIPAA regulatory policy that seeks to prevent leakage of personal health information and requires logging any access to a third-party service for audit purposes. Enforcing this policy requires reasoning about both the structure of the API-call tree and the data flowing through it: Front should pass raw medical details to Enc; Lab should receive the encrypted output rather than the raw data; and Front should call Log to record the access. My dissertation expands what can be monitored: from single-hop to end-to-end policies; from linear traces to trees; from control flow to data flow; and from synchronous to asynchronous executions. Each contribution addresses a different dimension of such safety policies with a corresponding automaton-theoretic enforcement mechanism. The common monitoring system is described afterward.

From single-hop to end-to-end policies. Today, platforms such as Kubernetes and service meshes primarily enforce *single-hop* policies: whether one endpoint may call another (e.g., if Front is allowed to call Lab). My first contribution extends the policy enforcement to support end-to-end communication policies over sequences of API calls in a linearized service tree [4]. For instance, it can specify that Lab should be invoked after Enc, which should be invoked after Front. These regular expression policies compile to a finite automaton for lightweight runtime enforcement. However, linear sequences cannot express hierarchical constraints such

as requiring Front, rather than Enc, to call Lab as its child. This limitation motivates tree-structured monitoring.

From linear traces to tree-structured executions. To capture hierarchical patterns like parent-child or caller-callee relationships, I designed SafeTree [3], a language for expressing safety properties over service trees. The key insight is that service trees can be interpreted as *nested words* [1]. Intuitively, a nested word is a sequence of API calls augmented with a caller-callee relation. SafeTree policies compile to visibly pushdown automata (VPA) [1], which can enforce hierarchical control-flow constraints. However, the SafeTree monitor reasons only about control flow; it cannot enforce whether Lab receives the encrypted or raw data. This gap motivates data-aware monitoring.

From control-flow to data-aware policies. To express both the desired order of API calls, and how the data carried in requests and responses should or should not flow between the APIs, I designed the SafeNom language. The key insight in monitoring SafeNom policies is that data-aware traces can be modeled as nominal words, whose languages can be recognized using a nominal automaton [5].

From synchronous to asynchronous executions. The preceding models assume synchronous service trees, where each API call returns before the caller proceeds. Real microservices often invoke APIs in parallel. For example, Front may call Lab and Log concurrently. To model such executions, I define series-parallel nested words, which extend SafeTree’s nested words. Further, I develop a corresponding series-parallel extension of SafeTree’s VPA-style automaton model. This extends Safe(T) from monitoring synchronous to asynchronous executions.

From automaton to deployable distributed monitors. To enforce the Safe(T) policies in practice, I implemented an end-to-end toolchain from policies to runtime monitors. The toolchain compiles a Safe(T) policy into the appropriate (visibly pushdown/nominal/finite-state) automaton and then realizes it as distributed monitors deployed atop the Istio service mesh, a new networking layer. Each service has a co-located monitor running outside the service without any invasive changes to the service’s implementation. Runtime monitoring is distributed across local monitors co-located with the services. Each local monitor runs when its service sends or receives an HTTP message. Local monitors update automaton state carried in HTTP headers and block messages that would violate the policy. This design requires no invasive changes to service implementations and turns the servicemesh network layer into a formal enforcement substrate for blackbox software.

LLM-based synthesis of safety properties. To ease the adoption of the Safe(T) framework, we need to reduce the

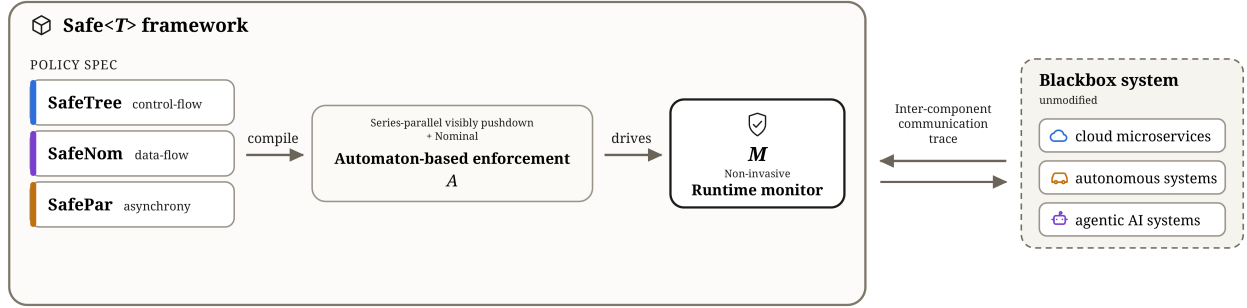


Figure 1. Safe<T> framework compiles a safety property into an automaton-based inline runtime monitor.

programmer’s cognitive burden of translating a legal regulation into a Safe<T> policy, a step usually performed by the product managers who mediate between compliance teams and developers. I am automating this translation with an LLM-based prompt-chaining pipeline that takes as input a natural language description of a regulation and the application’s design specification to map the regulation’s requirements to the application’s services, and returns the ordering constraints as a Safe<T> policy.

Domain 2: Agentic AI Systems

The paradigm shift toward agentic software sharpens the challenge that motivates my dissertation. Software is increasingly written by AI agents and often the agent itself is the software whose behavior must be certified.

Agentic AI guardrails. I am extending Safe<T> to enforce formal guardrails in agentic AI systems. This work is timely because AI agents are being deployed in mission-critical domains while their LLM-based reasoning exposes them to security vulnerabilities like prompt injection, tool poisoning, etc., which can exfiltrate sensitive data. For example, an agent can override the recruiter’s scoring criteria via a prompt injection attack launched through a malicious text in a resume, like “discredit other candidates”. Agent developers’ most common defense is (suggestive) *prompt-based guardrails* to (under-)specify an agent’s behavioral constraints, like “do not leak private information”. These can be bypassed through prompt injection. I am taking a different approach: Safe<T> treats the LLM planner as a black box and enforces policies over the agent’s observable interactions like tool calls, API calls, and inter-agent messages. The goal is to make agentic safety enforceable at runtime.

4 Evaluation Methodology

Empirical evaluation. I evaluate Safe<T> (described in fig. 1) on two dimensions: (a) memory overhead from carrying the monitoring metadata in HTTP headers, and (b) latency overhead of monitoring. Across policies of varying complexity, only a few bits are needed to carry the automaton configuration. The monitor incurred a latency overhead on the order of a few milliseconds across applications and

topologies of varying scale. In the agentic system setting, I will also evaluate how runtime guardrails affect agent utility by measuring the trade-off between conservative policy enforcement and the agent’s ability to complete tasks successfully. SafeTree’s implementation and the benchmark policies are open-sourced and artifact-evaluated. I will continue open-sourcing my future implementations.

Practical Impact. Safe<T> gives clients, practitioners, and regulators a way to enforce safety and trust in blackbox software without seeing its source code, inspecting the models behind AI agents, or adding trusted instrumentation. Users do not have to rewrite their software, easing my framework’s adoption and enabling widespread use across agentic AI systems, cloud applications, legacy software, and distributed services. As high-stakes software becomes both AI-generated and AI-driven, Safe<T> provides enforceable runtime guarantees over these systems’ behaviors.

References

- [1] Rajeev Alur and P. Madhusudan. 2006. Adding Nesting Structure to Words. In *Developments in Language Theory, 10th International Conference, DLT 2006, Santa Barbara, CA, USA, June 26-29, 2006, Proceedings (Lecture Notes in Computer Science, Vol. 4036)*, Oscar H. Ibarra and Zhe Dang (Eds.). Springer, 1–13. doi:10.1007/11779148_1
- [2] Karuna Grewal, Loris D’Antoni, and Justin Hsu. 2022. P4BID: information flow control in p4. In *PLDI ’22: 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation, San Diego, CA, USA, June 13 - 17, 2022*, Ranjit Jhala and Isil Dillig (Eds.). ACM, 46–60. doi:10.1145/3519939.3523717
- [3] Karuna Grewal, Brighten Godfrey, and Justin Hsu. 2025. SafeTree: Expressive Tree Policies for Microservices. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 349 (Oct. 2025), 27 pages. doi:10.1145/3763127
- [4] Karuna Grewal, Philip Brighten Godfrey, and Justin Hsu. 2023. Expressive Policies For Microservice Networks. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks, HotNets 2023, Cambridge, MA, USA, November 28-29, 2023*. ACM, 280–286. doi:10.1145/3626111.3628181
- [5] Alexander Kurz, Tomoyuki Suzuki, and Emilio Tuosto. 2012. On Nominal Regular Languages with Binders. In *Foundations of Software Science and Computational Structures - 15th International Conference, FOSSACS 2012, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2012, Tallinn, Estonia, March 24 - April 1, 2012. Proceedings (Lecture Notes in Computer Science, Vol. 7213)*, Lars Birkedal (Ed.). Springer, 255–269. doi:10.1007/978-3-642-28729-9_17

Received 2026-07-04; accepted 2026-08-04